

# Tailoring Design for Embedded Computer Vision Applications

**Jason Schlessman**, adidas Digital Sports

**Marilyn Wolf**, Georgia Tech

**C**omputer vision algorithms have unique requirements that do not align smoothly with the efficiency and cost of embedded systems. On one hand, computer vision algorithms require huge amounts of computational power; on the other, there is no single universal computer vision pipeline.

One stable set of algorithms that is suitable for all applications would enable the careful design of a single platform that could serve a wide range of computer vision systems. However, different applications use different algorithmic combinations, often with divergent parameters, so designers must conduct analyses to match application needs to the architectural platform.

Over the past decade, our research has explored platform design requirements and processes for embedded computer vision systems in depth. As part of our work, we developed a methodology for design space exploration that allows designers to quickly evaluate hardware and software tradeoffs, determining both the overall computing volume needed and the computing units' specific characteristics.

Our methodology focuses on two critical challenges in computer vision systems: numerical characteristics and memory. Each of these tasks is important independently, but they also interact because high-precision numerical representations require more memory and

*Numerical dynamic range analysis combines with architectural simulation to help designers explore algorithmic and architectural tradeoffs that can meet embedded computer vision's demand for precision and high-memory bandwidth.*

memory bandwidth. Our methodology combines numerical dynamic range analysis with computer architecture simulations to aid designers in algorithmic and architectural codesign and redesign, as well as provide insights that can guide them to a more informed platform choice.

We used our methodology for several designs, including a smart camera-based surveillance system that monitors people in train stations.<sup>1</sup> The ability to perform in-camera analysis is important for large-scale computer vision systems. Some multicamera systems send video to the cloud or a remote server for analysis, which consumes significant bandwidth.<sup>2</sup> In-camera analysis systems can ensure that raw video never leaves the camera; the additional privacy created by the lack of a video record is attractive in many situations. However, building a distributed smart-camera system requires nodes with sufficient computational power to implement the required algorithms.<sup>3,4</sup> The design space of multicamera computer vision systems is even larger, further motivating the need for customized computational platforms.

## OUR METHOD ADDRESSES THE LARGE SEMANTIC GAP BETWEEN ALGORITHM DESIGN AND ARCHITECTURAL CONCERNS.

With our methodology, multicamera system designers can quickly compare field-programmable gate array (FPGA) and CPU platforms as well as graphical processing units (GPUs), which makes hardware–software codesign more efficient. Such comparisons are becoming increasingly important in assessing system efficacy, particularly in GPU-based systems.

### CURRENT TOOL LIMITATIONS

Our design methodology is aimed at the large semantic gap between the high level at which most computer vision algorithms are designed and the concerns that architects must address to build a viable embedded computer vision implementation. Many algorithm designers liberally use double-precision, floating-point arithmetic to avoid dealing with numerical problems, which incurs substantial memory and energy costs. They often use Matlab or the OpenCV library, which provide library functions for very abstract operations, obscuring implementation costs.

Memory bandwidth and size, real-time performance, and energy and power consumption—key metrics for judging an embedded computing system’s value—are not easily discernible from these abstract representations. Even if an embedded system specialist creates the final algorithm, computer vision algorithm designers should be actively involved in exploring design tradeoffs among accuracy, performance, and energy use.

Computer vision algorithm designers are better equipped for this design space exploration because most current embedded systems tools do not have enough abstraction. Consequently, considerable effort is required to

sufficiently translate the algorithm to evaluate design tradeoffs that can lead to high accuracy, low execution times, and efficient energy use. For example, the Xilinx System Generator and Jem-Vivado design suites<sup>5,6</sup> provide digital signal processing (DSP) blocks and high-level synthesis blocks, but these tools require architecture and hardware design expertise. Work on DSP-oriented abstractions such as synchronous data-flow provides a higher-level programming abstraction, but tools for automatically mapping algorithmic tasks to heterogeneous architectures are still lacking. Such tools can greatly streamline the design process. For example, in previous work,<sup>7</sup> we developed methods for mapping memory-access patterns onto banked and ranked dynamic RAM (DRAM) structures as they were given—tasks that generally involve manual work.

Matlab Fixed Point Designer<sup>8</sup> provides tools for analyzing numerical accuracy requirements but does not relate those characteristics to real time-oriented characteristics such as memory bandwidth. Architectures have been designed for particular operations, such as EFFEx for feature extraction,<sup>9</sup> but these architectures provide only a single point in the design space.

### PLATFORM-NEUTRAL EXPLORATION

Our methodology exploits platform-neutral tools that measure algorithmic characteristics, which can help identify the algorithm’s key bottlenecks. Many designers of high-performance embedded systems first map the algorithm to a platform and then optimize it; our methodology emphasizes using

platform-neutral measurements to guide platform selection.

### Algorithmic characteristics

Analysis begins by implementing a prototype computer vision algorithm, which is to be mapped to an embedded system on a general-purpose PC. To determine the software implementation’s complexity, we analyze its constituent tasks by splitting the system into relevant tasks with operations.

We then measure the algorithm’s resource requirements, including number of operations and memory bandwidth. Although back-of-the-envelope analysis is adequate for simple algorithms, many computer vision algorithms have complex control flows that preclude manual analysis. We can obtain the required measurements without fully mapping to the target implementation, generating operation and memory-access counts on one architecture and then mapping those counts onto others. Measuring operation counts and memory access on code close to the algorithm designer enables very early design space exploration, which in turn allows algorithmic tailoring if necessary.

Tools such as SimpleScalar<sup>10</sup> or VTune (<http://software.intel.com/en-us/intel-vtune-amplifier-xe>) are useful in exploring executable code. If the algorithm was designed in a language such as C or C++, SimpleScalar or VTune can operate on the compiled code; for algorithms designed in Matlab, designers can generate code and then use these tools to analyze it.

### Hardware requirements

Once designers have established the resource requirements, they must

consider hardware needs: number of processors, special-purpose accelerators, and so on. Resource measurements give direct information on throughput and energy consumption, but not as much data on latency. This characteristic is better addressed through dataflow analysis, which is independent of metrics such as cache size and direct memory access (DMA).

To facilitate platform choice, designers can perform additional simulations to measure throughput and energy consumption. In some cases, designers might need to refine the original task decomposition to better align with accelerator availability or to accommodate a particular function's implementation in hardware. For example, designers might opt to implement a kernel function in hardware but use software to control the kernel's execution.

Many platforms allow some latitude in choosing numerical representation outside of data type specification (char, integer, or double). This is particularly true for FPGAs but also—to a lesser extent—for DSPs and microcontrollers. As a result, designers can change representation and do not need to rely solely on single or double precision.

The design of numerical representations and accuracy analysis is widely recognized as a key part of DSP hardware and software design. Analyzing the application's permissible change in output accuracy with respect to savings in bandwidth and internal storage is a critical part of design space exploration. Clearly, the choice of representation has a direct impact on internal memory use and bandwidth. Furthermore, because computer vision algorithms, like any computationally

intensive task, can require large amounts of internal storage, identifying memory requirements early on allows algorithmic tailoring that can save design effort at the later, more expensive implementation stages.

## APPLICATION

We used our methodology with two subsystems that are common in computer vision applications: optical flow-based motion analysis and background subtraction based on the mixture-of-Gaussian (MoG) technique.

### Optical flow measurement

Designers often use the Kanade-Lucas-Tomasi (KLT) optical flow algorithm<sup>11</sup> to analyze motion. The algorithm is computationally intensive, requiring significant memory bandwidth, as well as numerical precision in matrix inversion and attendant division.

To design an embedded system for the KLT algorithm's core,<sup>12</sup> we followed our methodology's first step, which is to identify the algorithm's constituent tasks. The tasks are embodied in the algorithm's overall goal: to determine eigenvector  $\langle U_x, U_y \rangle$  from successive image gradients  $G_x, G_y, G_t$ . Tasks include:

- › Apply convolution filters on successive image windows  $x, y$  to determine the gradients  $G_x, G_y, G_t$  as well as the gradient matrix  $\mathbf{A} = \langle G_x, G_y \rangle$ .
- › Perform matrix multiplication and inversion to determine eigenvectors  $\langle U_x, U_y \rangle = (\mathbf{A}^T \mathbf{A})^{-1} * \mathbf{A}^T G_t$ , which entails four steps:
  - › compute  $\mathbf{C} = (\mathbf{A}^T \mathbf{A})$ ,
  - › determine  $\mathbf{C}^{-1}$ ,
  - › compute  $\mathbf{D} = (\mathbf{A}^T G_t)$ , and
  - › compute  $\langle U_x, U_y \rangle = \mathbf{C}^{-1} * \mathbf{D}$ .

We then had to decide if the representation should be based on integer or rational values and, if rational values, to determine if fixed- or floating-point values were more suitable. We noted that after converting pixel intensity, the KLT algorithm calculates gradients for a given frame and then performs computations on the frame's subwindows. These tasks require rational numbers, division, square root, and matrix operations, which quickly motivated us to concentrate on an FPGA-based implementation.

Relying on rational numbers for precision is not unique to the KLT algorithm. Indeed, many embedded systems rely on fixed-point representation. In reconfigurable fabrics, synthesis tools can more intuitively access designs based on fixed-point representation. These designs also eliminate the normalization logic overhead inherent in designs based on floating-point representations.

We used software simulations to analyze the KLT algorithm's requirements for numerical accuracy, after discovering the importance of carefully considering numerical properties. The algorithm's pixel inputs and motion vector outputs are relatively small, and initially we thought that low-precision arithmetic would be sufficient. However, an examination of the steps that transform the pixels into motion vectors revealed the need for greater accuracy. To illustrate, consider the extreme case of 255 values for incoming RGB pixel intensities. A gradient  $G$  could have a maximum element of  $\text{window\_size} * (2 * 255)$ , which would require  $\log_2(\text{window\_size} * 510)$  bits. Our chosen pixel window size of  $9 \times 9$  would then mandate 16 magnitude bits. Because the FPGAs we considered

## METHODS LIKE OURS SHOULD SEE WIDER USE AS VISION AND SIGNAL PROCESSING RELY MORE ON FLOATING-POINT COMPUTATION.

provide only 18-bit-square multipliers, a mere one fractional bit would be available. Moreover, the later computation of  $\mathbf{A}^T\mathbf{A}$  would require additional magnitude bits.

A floating-point representation would provide greater dynamic range and fractional precision without incurring a severe bit-width penalty. Naturally, this flexibility does not come without a price, since floating-point numbers require normalization logic (fractional point matching) as well as exponent modifications.

The results of this design space exploration led us to choose a representation format that we can best describe as a modification of the IEEE 32-bit single-precision floating-point standard. In this standard, the range of significand values (significant digits of a floating-point number) is  $(-2, 2)$  requiring one magnitude bit; all other bits are devoted to fractional representation. The exponent portion represents the power of 2 by which the significand is multiplied when interpreting the data value. Finally, each number contains a sign bit. Consequently, unlike fixed-point representation, we do not require signed arithmetic units. The multipliers in our target FPGA are square multipliers when used with unsigned values. Our modified representation consists of floating-point values with reduced-width significands, 6-bit exponents, and a sign bit, enabling a total of 24 bits to represent the intermediate and resultant data.

### Background elimination

Designers use background elimination to extract relevant portions of a scene, labeling components as either of interest (foreground) or preexisting (background). The background modeling

approach is based on an augmented Adaptive MoG model approach.<sup>13</sup> It provides a useful contrast to the KLT algorithm because MoG-based background elimination has particularly high memory requirements.

These requirements stem from the algorithmic approach of building several models of each pixel's Gaussian statistics. It then continuously evaluates the models to determine foreground and background.

To eliminate the background, the Adaptive MoG algorithm performs four tasks for each color channel of each pixel:

- › computes the mean and covariances over the frame's pixel windows;
- › matches metrics to the Gaussian model;
- › updates the models' Gaussian mean and variance; and
- › ranks current Gaussian models with respect to successful matches.

To determine dynamic range and precision requirements, we performed numerical analyses on the background subtraction PC code, similar to the way we conducted our optical flow analysis in the previous application example. We determined that most intermediate values could be relegated to floating-point values represented as shifted 16-bit integers, while the remaining values would fit within shifted 32-bit integers. We made this determination after using ground-truth video data that we processed through the reduced representation form and obtaining a match of 98.7 percent correspondence with a PC-based implementation. We designed both hardware-oriented FPGA-based

and software-oriented system-on-chip implementations.

Our software implementation yielded significant insights. On the basis of instruction count and the numerical analysis of our Adaptive MoG code, we estimated that a processor speed of 60 MHz would meet our target frame rate. We selected an ARM Cortex M3, which achieves this rate, and ported our generic PC-based background subtraction code (baseline) to the STM32F103, which has an M3 core. We then analyzed implementation performance in more detail, using the IAREmbedded Workbench for ARMCSpy simulator ([www.iar.com](http://www.iar.com)) as well as a J-Trace profile tool on an ST STM3210E-EVAL target evaluation board.

Table 1 shows the results of profiling two implementations. The first is an implementation we developed using a direct method to convert double-precision Matlab code to floating-point C code using no other numerical or bandwidth analysis. The second is an implementation developed using our methodology, beginning with the same Matlab code and ending with C code.

As the table shows, the design is feasible on the STM32F103 target using our methodology. However, the direct method would not only exceed the target's memory capacity, but it would also usurp all its CPU cycles while still having to run at a reduced frame rate. In fact, the direct method would have required a Cortex M4 with a dedicated floating-point unit, assuming it could even be mapped to a Cortex. Even if the direct method could use slightly fewer CPU cycles because of the M4's arithmetic acceleration, that acceleration would come with a commensurate increase in power consumption, device footprint, and cost.

**TABLE 1.** Results for two implementations of background subtraction on the STM32F103 microcontroller.

| Algorithm mapping            | Number of instructions | Number of floating-point instructions | SRAM use (Kbytes) | CPU speed (MHz) |
|------------------------------|------------------------|---------------------------------------|-------------------|-----------------|
| Matlab to C: direct method   | 50,000                 | 48,000                                | 240               | 130             |
| Matlab to C: our methodology | 90,000                 | 0                                     | 70                | 63              |

## ABOUT THE AUTHORS

**JASON SCHLESSMAN** is a senior manager in the Data Science and Analytics group of adidas Digital Sports Futures Division. While conducting the research described in this article, he was a graduate researcher in Princeton University's Electrical Engineering Department. His current research interests include data science as applied to athletic performance and human wellness, firmware development for tracking soccer players' performance, embedded computing and computer vision, and multiprocessor-embedded systems. He is a member of IEEE. Contact him at jschlessman@gmail.com.

**MARILYN WOLF** is a Farmer Distinguished Chair and Georgia Research Alliance Eminent Scholar at Georgia Tech. Her research interests include cyber-physical systems, embedded computing, embedded video and computer vision, and VLSI systems. Wolf received a PhD in electrical engineering from Stanford University. She is a Golden Core member of the IEEE Computer Society, a Fellow of IEEE and ACM, and a recipient of the ASEE Terman Award and IEEE Circuits and Systems Society Education Award. Contact her at wolf@ece.gatech.edu.

Computer vision is a leading-edge application for embedded computing, requiring high-performance, high-accuracy computation as well as high-bandwidth memory access. The platform-neutral exploration of alternatives can illuminate algorithmic and architectural tradeoffs, allowing designers to arrive at efficient implementations in a timely fashion. We believe that some aspects of methodologies such as ours will see wider use as high-end signal processing begins to rely more heavily on full single- and double-precision floating-point computation. As computer vision algorithms mature, they will need new generations of specialized design methodologies, making the insights gained from design space exploration even more valuable. 

## ACKNOWLEDGMENTS

This work was funded as part of National Science Foundation grants 0509463 and 0325119.

## REFERENCES

1. J. Schlessman et al., "Heterogeneous MPSoC Architectures for Embedded Computer Vision," *Proc. IEEE Int'l Conf. Multimedia and Expo (ICME 07)*, 2007, pp. 1870–1873.
2. G. Hua et al., "Introduction to the Special Issue on Mobile Vision," *Int'l J. Computer Vision*, vol. 96, no. 3, 2012, pp. 277–279.
3. R. Tron and R. Vidal, "Distributed Computer Vision Algorithms," *IEEE Signal Processing*, vol. 28, no. 3, 2011, pp. 32–45.
4. H. Aghajan and A. Cavallaro, *Multi-Camera Networks: Principles and*

- Applications*, Academic Press, 2009.
5. *Vivado Design Suite Reference Guide: Model-Based DSP Design Using System Generator*, UG958 (v1012.3), Xilinx, 16 Nov. 2012.
6. *JemVivado Design Suite User Guide: High-Level Synthesis*, UG902 (v2012.2), Xilinx, 25 July 2012.
7. D. Lee, M. Wolf, and S.S. Bhattacharyya, "High-Performance and Low-Energy Buffer Mapping Method for Multiprocessor DSP Systems," *ACM Trans. Embedded Computing Systems*, vol. 12, no. 3, 2013, p. 82.
8. Mathworks, "Fixed-Point Designer MATLAB," Jan. 2014; [www.mathworks.com/products/fixed-point-designer](http://www.mathworks.com/products/fixed-point-designer).
9. J. Clemons et al., "EFFEX: An Embedded Processor for Computer Vision Based Feature Extraction," *Proc. 48th ACM Design Automation Conf. (DAC 11)*, 2011, pp. 1020–1025.
10. D. Burger and T. Austin, "The SimpleScalar Toolset, Version 2.0," *ACM-SIGARCH Computer Architecture News*, vol. 25, no. 3, 1997, pp. 13–25.
11. B.D. Lucas and T. Kanade, "An Iterative Image Registration Technique with an Application to Stereo Vision," *Proc. Int'l Joint Conf. Artificial Intelligence (ICJAI 81)*, 1981, pp. 674–679.
12. J. Schlessman et al., "FPGA-Based Design of a Surveillance System Employing Optical Flow," *Proc. Workshop Synthesis and System Integration of Mixed Information Technologies (SASIMI 06)*, 2006, pp. 110–116.
13. T. Horprasert, D. Harwood, and L.S. Davis, "A Statistical Approach for Real-Time Robust Background Subtraction and Shadow Detection," *Proc. IEEE Int'l Conf. Computer Vision Frame-Rate Workshop*, 1999, pp. 1–19.